

Reducing Computational Cost in Soybean Leaf Disease Classification with Neural Networks

Redução de Custo Computacional na Classificação de Patologias em Folhas de Soja Usando Redes Neurais

Abstract: Meeting the growing global demand for soybeans requires increased agricultural production. Brazil, as the largest soybean producer, faces substantial losses due to the underdiagnosis of foliar diseases. Given the economic importance and the challenges involved in diagnosis with traditional methods, there is a strong potential to automate this process through computer vision and artificial intelligence. Recent approaches often rely on more complex Convolutional Neural Networks, which limit the accessibility and scalability. To promote broader access, this work explores lightweight architectures, specifically MobileNet, SqueezeNet, ResNet-18, ResNet-50 and ResNet-101 to compare their performances to heavier models. While lightweight models underperformed in accuracy compared to deeper architectures, they demonstrated competitive performance with reduced computational costs, reinforcing their applicability in resource-constrained scenarios. The computational efficiency of these models drove the development of a prototype of a Android application, increasing accessibility to automated disease diagnose in the field.

Keywords: Computer Vision — Plant Disease Classification — Convolutional Neural Networks — Computational Efficiency

Resumo: Atender à crescente demanda global por soja exige um aumento na produção agrícola. O Brasil, como maior produtor de soja, enfrenta perdas substanciais devido ao subdiagnóstico de doenças foliares. Dada a importância econômica e os desafios envolvidos no diagnóstico por métodos tradicionais, há potencial para automatizar esse processo por meio de de visão computacional e inteligência artificial. Abordagens recentes frequentemente dependem de Redes Neurais Convolucionais mais complexas, limitando a acessibilidade e escalabilidade. Para promover maior um acesso, este trabalho explora arquiteturas, especificamente MobileNet, SqueezeNet, ResNet-18, ResNet-50 e ResNet-101 a fim de as comparar com modelos mais pesados. Embora as arquiteturas leves tenham apresentado acurácia inferior em relação às mais profundas, elas demonstraram resultados competitivos com custos computacionais reduzidos, reforçando sua aplicabilidade em cenários com restrição de recursos. A eficiência computacional norteou o desenvolvimento de um protótipo de aplicativo Android, ampliando o acesso ao diagnóstico automatizado no campo.

Palavras-Chave: Visão Computacional — Classificação de Doenças em Plantas — Redes Neurais Convolucionais — Eficiência Computacional

DOI: <http://dx.doi.org/10.22456/2175-2745.XXXX> • Received: dd/mm/yyyy • Accepted: dd/mm/yyyy

CC BY-NC-ND 4.0 - This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

1. Introduction

Computer Vision is a field of computer science that allows computers to “see” and interpret images. One of its relevant applications is image classification through the use of Convolutional Neural Networks (CNNs) alongside artificial intelligence. CNNs successfully analyze images, as they can identify complex visual patterns and features. This capability makes them a great tool for the automated detection of plant diseases, a challenge that has a significant impact on agriculture.

A survey by Wrather and Koenning [1] indicates that

Brazil loses, on average, from 10% to 20% of soybean (*Glycine max*) production annually due to diseases. With the objective of mitigating these losses in soybean cultivation, the methods of diagnosing foliar pathologies have undergone a reinvention.

In this context, considering that human detection of these anomalies is a costly process (as it requires specialized labor) and, at times, inefficient, there is potential for automated solutions based on computer vision and artificial intelligence. Through this capability, several studies have been conducted using CNNs, mostly employing deep architectures with an excessive number of parameters. These characteristics generally

provide significant improvements in the speed and accuracy of detection.

Despite promising results using deep convolutional neural networks for the effective classification of foliar diseases, most applications require great computational capacity, making the adoption of these methods difficult for producers with limited resources. With this in mind, it becomes relevant to seek alternatives of lighter architectures that lead to results similar to those presented by more complex projects but with reduced cost to improve broader accessibility to automated tools.

This work investigates lighter and shallower networks capable of obtaining results comparable to those of more complex architectures, with the aim of reducing the complexity in the application of automated methods for early diagnosis. Furthermore, the proposal may enable the generation of statistical data on the crop, such as mapping the frequency and location of pathologies, which can support targeted treatment and more efficient agronomic decisions.

In addition to the investigation of lighter architectures, the proposed work also aimed to develop a prototype of a mobile application to certify the usability of those simpler networks. The primary goal of the app is to demonstrate the benefits of using lightweight CNNs in a more common and accessible operation. The main recommendation for its use is the premature diagnosis of foliar pathologies directly in the field.

This on-device processing is crucial to farmers, as it provides a solution to the common challenges of fast identification of diseases that can hinder their productivity. The application performs a fast analysis without requiring an Internet connection, since rural areas often have limited connectivity. This real-time feedback allows farmers to quickly identify the problem without the need for practical methods. These methods often involve collecting leaf samples, which may require destroying the plant, and sending them to a laboratory for analysis by a specialized professional. This approach saves time and transportation resources, while also enabling greater control over plant treatments, leading to increased productivity.

Moreover, through this efficient classification, the generation of statistical data on crop health can provide support and guidance in the effective treatment of diseases that most affect production, contributing to increased quality and productivity. Therefore, this study seeks to fill a gap by exploring more accessible solutions without compromising effectiveness.

2. Related work

The image classification problem applied to plant pathologies is a well-known and explored subarea of computer vision, given its broad application in various agricultural contexts. In this domain, significant advances have been made using artificial intelligence and computational resources. Akshai *et al.* [2] conducted an analytic survey on the use of deep learning for plant disease classification, focusing on how deep and more complex architectures performed. Similarly, in a

study that served as main reference for our work, Bevers *et al.* [3] applied neural networks to specifically analyze soybean pathologies. Although these projects aid in solving the main problem using deep learning techniques, they are not fairly applicable in a resource-constrained scenario as they often rely on computationally intense models.

This limitation has led to a growing interest in simpler solutions with similar efficiency. Parallel to this work, some studies have sought to reduce the computational cost of classification applications to increase access to efficient automated tools. Peyal *et al.* [4], concomitantly to the proposed work, also explored lighter architectures to classify cotton and grape plant diseases and demonstrated that using those networks it is possible to achieve accurate results. Hui Fuang *et al.* [5] developed a mobile application to detect foliar diseases in an environment without Internet access, emphasizing the importance of on-device processing and accessibility for real-world use.

This study builds on the foundation of these works. Although the mentioned researches and studies have proven the efficacy of deep learning to classify plant anomalies and demonstrated the potential of lightweight models and mobile applications, a gap remains. There is a lack of comprehensive analysis that evaluates the benefits and performance of less complex models, especially for soybean foliar diseases, in a context geared toward on-device mobile applications. Therefore, the proposed work aims to fill this gap by exploring classical lightweight CNNs to find a balance between accuracy and computational efficiency for an accessible applicability.

In order to explore the effectiveness of lightweight neural networks for the classification of soybean disease, three distinct image datasets were chosen. The ASDID dataset was selected as the primary data source because it was the same as the one used and created by Bevers *et al.* [3], a reference work in the field of classification of soybean anomalies. This choice allows a direct comparison between the results of the lighter networks in this work and the deeper approaches explored in the other study. To broaden our analysis, increase the number of information for training, and evaluate the robustness of the model, we also included the SoyNet [6] and SoyBean Disease (a collaborative database) datasets, which feature binary and multiclass classification problems, respectively. The use of multiple data collections ensures a more effective evaluation of our proposed approach and allows different processes and methods in the training period.

3. Methodology

The proposed work was carried out in the following five stages.

3.1 Architectures Selection

For this project, five CNNs were chosen in order to reduce computational cost in the task of classifying soybean leaves. Heavier architectures such as ResNet-101 (44.5 million parameters) and ResNet-50 [7] (25.6 million parameters) were



Figure 1. Samples from the datasets. From left to right: *SoyNet Healthy*, *SoyNet Disease*, *SoyBean Bacterial Blight*, and *ASDID Potassium Deficiency*.

used as comparative benchmarks, despite their complexity and depth. To reduce the computational cost of networks to classify foliar soybean diseases, which is the primary goal of this study, this work highlights the performance and results of lighter networks, such as ResNet-18 [7] (11.7 million parameters), an intermediate network in terms of depth and complexity that served as a starting point for cost reduction. Additionally, SqueezeNet [8] (1.25 million parameters) was explored, known for its drastic reduction in size and depth, and MobileNet [9] (4.2 million parameters), which was specifically developed by Google for mobile applications.

3.2 Fine-Tuning

The SoyNet dataset used in this stage contains over nine thousand high-quality soybean images, encompassing healthy and diseased leaves. These images were captured from various angles and directly sourced from soybean agriculture fields. The collection contains both raw and processed images with color alterations and is sectioned into two different classes, which allows training for binary problems.

Firstly, to adapt the parameters and weights of our networks to the specific soybean leaf problem, a fine-tuning process was performed in each architecture using the SoyNet dataset. In this way, the models were able to solve a binary classification and distinguish healthy leaves from diseased ones with more precision.

This process was executed using networks pre-trained with ImageNetV1 [10] as the entire architecture was subjected to a freezing process, except for the last feature extraction block and the classification layer. Therefore, the model adjusted its weights and training parameters to the ImageNet multiclass problem.

3.3 Transfer Learning

Sequentially to the initial fine-tuning, the models were further adapted to solve the multiclass classification problem. A transfer learning approach was applied to the fine-tuned networks on the SoyNet dataset in order to train them on a more granular SoyBean and ASDID datasets.

The SoyBean dataset was the first one used in this stage. Although it contains only a small number of images (around

one thousand), it was chosen to adjust the models' hyperparameters, such as the optimizer, batch size, and learning rate, on a complex task with limited data. This collection was primarily used to facilitate the fine-tuning of these parameters, especially the optimizer, since training with SoyBean consumes significantly less computational power than with the ASDID dataset. By identifying the best combination of variables with this smaller dataset, we were able to make the subsequent, more computationally intensive training on the ASDID dataset more efficient and effective.

The ASDID dataset, our primary collection, was used next, as it presents a significantly larger volume of data for multiclass classification and captures a greater diversity in environmental conditions and backgrounds. With approximately ten thousand high-quality raw images, which later were resized to a final dimension of 226x226, ASDID was a crucial part of our evaluation, as it was also used in a reference work by Bevers *et al.* [3], allowing a direct comparison of our results.

This process was performed by freezing all layers except the final classification layer, which was modified to output the specific number of classes of each dataset. By training only the final layer, the networks were adapted to the new and more complex classification tasks while avoiding the need to train all parameters. This approach significantly reduced training time and computational costs. Furthermore, this step allowed us to evaluate the models and their ability to generalize from a simple binary problem to more challenging multiclass problems, which is critical for real-world applications.

3.4 Conversion to PyTorch Lite

After training the CNN models, it was necessary to convert them from the ".pkl" format to the ".plt" Pytorch Lite format. This conversion was essential for the models' implantation in the Android application, which allowed real-time, on-device inference without an internet connection. This a critical step for our main goal: to increase accessibility to reliable automated diagnosis tools for soybean producers in the field, where internet is limited or even unavailable.

The conversion was performed through a Python script. First, the original model was loaded and put in assessment

Table 1. Dataset partition

Dataset	Train	Test	Total
SoyNet	5,109	2,190	7,299
SoyBean	816	351	1,167
ASDID	7,024	3,011	1,035

mode, then, the model was converted to the TorchScript format, which is optimized and prepared for mobile applications and can be executed by the Lite Interpreter, a lighter and faster Pytorch version, ideal for devices with limited resources. Finally, the converted model was saved and linked to the application's assets, ready to use.

3.5 Android application

A Android application was developed to validate and test the performance and applicability of lightweight architectures. It is structured into two main components: "Classifier.java" class and a "MainActivity.java" activity.

The Classifier class is responsible for all interactions with the PyTorch model. It contains the necessary methods and utilities from PyTorch for loading the model and its labels, along with processes such as resizing, normalizing, and converting the input image into a tensor format. It also handles the prediction process and the representation of the result's confidence score using SoftMax.

On the other hand, the "MainActivity" activity orchestrates all applications' functionalities, considering that it possesses the script of the activity done by the user, such as calls of the Classifier class's methods, search of images in the gallery or of photos taken by the camera, and return of the prediction and its confiability made to the user.

4. Experiments

The experiments were carried out with the five architectures initially chosen: MobileNet, SqueezeNet, ResNet-18, ResNet-50, and ResNet-101. The training methodology was sectioned into two main stages: initial fine-tuning on a binary dataset, followed by transfer learning on multiclass datasets. For all methods, three different datasets were used and their partition is detailed in Table 1.

All experiments were conducted using Python 3 and the Pytorch deep learning framework, using a NVIDIA GeForce RTX 4070 GPU. To develop the prototype of an Android application, we used tools such as Android Studio, and to convert the trained models into compatible formats with the application, we utilized the PyTorch Mobile framework.

The first step used the SoyNet dataset and executed a fine-tuning in all architectures already pre-trained with ImageNetV1 weights as it prepared and shaped the parameters of the neural networks to receive soybean leaf images in a more complex problem, since this first obstacle consists of a binary separation between healthy and diseased plants, used only for adaptive purposes.

Following this initial process, transfer learning techniques were applied to train the same networks using the weights of the resulting models. In this context, the SoyBean and ASDID datasets were responsible for providing more complex multiclass training. Thus, these subsequent trainings used the adjustments resulting from the binary problem to further dive into the classification of specific foliar diseases, a challenging task, since the differences between each disease depend on the minutiae and details of each image.

In parallel to this sequential methodology, various experiments were conducted to optimize the models' performance. The initial training runs were executed using the parameters detailed in Table 2, which presented reasonable results. The choice of parameters was based on the reference work by Bevers *et al.* [3], aiming to demonstrate that manipulating similar parameters with a different approach yields promising results.

To further improve the accuracy of our models, several variables were systematically altered to verify which combination performed better. These variables included batch size, optimizer, learning rate, and, in some experiments, the number of epochs. However, it was found that no significant changes occurred once a minimum of 100 epochs were reached. These experiments to adjust the hyperparameters were executed with the SoyBean dataset, as training with it is less computationally demanding than with the larger ASDID dataset. This preliminary step was a time-saving strategy that allowed our main trainings to be executed with better parameters combination. The results of those alterations are discussed in the following subsections.

4.1 Results

Good results can be observed in the fine-tuning stage. Although it is a simple problem, it is important to highlight that, given the model adjustment for the task, the results in all architectures were above 94%, as detailed in Table 3 and Figures 2 and 3.

Furthermore, it is possible to analyze that for a more complex problem, such as the multiclass one, in which transfer learning techniques were performed, satisfactory accuracies were achieved, reaching values of up to 80% in the best models, as it can be seen in Table 4. A more in-depth analysis of the results with the SoyBean dataset, used to adjust hyperparameters, revealed the superior effectiveness of the Adam optimizer for this specific task. Adam outperformed the other experimented SGD optimizer across different architectures, as can be seen in Table 4 and in Figures 4 and 5. Additionally,

Table 2. Base parameters

Parameter	Value
Batch	32
Epochs	100
Learning Rate	0.0001
Loss Function	Cross Entropy

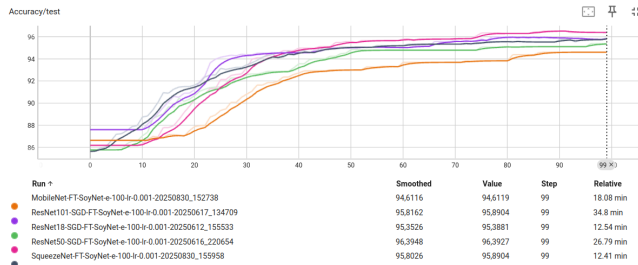


Figure 2. Accuracy curve on the test set in TensorBoard (Fine Tuning with SoyNet).

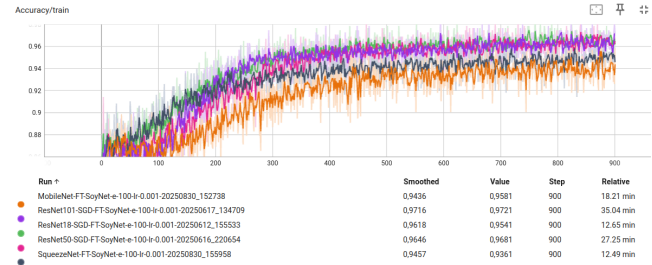


Figure 3. Accuracy curve on the train set in TensorBoard (Fine Tuning with SoyNet).

Table 3. Fine-Tuning with SoyNet Results

Architecture	Optimizer	Dataset	Accuracy
MobileNet	SGD	SoyNet	94.61%
SqueezeNet	SGD	SoyNet	95.89%
ResNet18	SGD	SoyNet	95.39%
ResNet50	SGD	SoyNet	96.53%
ResNet101	SGD	SoyNet	95.98%

this is the reason why the Adam optimizer was chosen to carry the more demanding trainings with ASDID, as it can be observed in Table 5.

Given these findings, to optimize our computational resources further experiments with all architectures were only

conducted using the Adam optimizer. This decision was based on the fact that Adam's learning rate properties make it effective for optimizing a wide range of parameters, which excluded the need to explore less effective optimizers like SGD. By not using the SGD optimizer, we were able to significantly reduce our overall training time while focusing on the most promising optimization strategy for our lightweight models.

In summary, the results were satisfactory, given the differences in depth and number of parameters of the architectures. We highlight that MobileNet, SqueezeNet and ResNet-18 architectures, which were the main focus of this work, presented significant results, having as an advantage the cost-benefit in terms of computational cost and training time. By using the same dataset as the reference study by Bevers *et al.* [3] we still obtained promising accuracies, as more clearly detailed in Table 5 and in Figures 6 and 7.

Finally, to validate the real-world applicability of the proposed lightweight models and to base this study, a prototype of an Android application was developed. This application serves as practical tool for testing the efficiency of this models in a real-use scenario. The app was evaluated on a Samsung S24 mobile device with a, Exynos 2400 processor to simulate a more popular operational system. The results showed that the inference speeds on the mobile device were very similar to those on the computer, with imperceptible time differences.

The mobile app is able to either access the phone's gallery or use the device's camera to perform classification, with an inference time of approximately 200ms. After this processing, it displays the leaf diagnosis and the corresponding classification accuracy. The main objective of this application is to demonstrate the practical deployment on mobile devices and benefits of lightweight neural networks to classify images of soybean plants with anomalies, highlighting their potential for accessible and efficient field diagnosis.

The proposed Android application's layout is a simple interface designed to test the performance of lightweight models on a mobile device. To improve accessibility, the app operates without an internet connection and allows users to access the device's camera or photo gallery via simple, intuitive buttons, as shown in Figure 8. After the diagnosis, the results, as well as the accuracy and time inference are displayed at the bottom.

Table 4. Transfer Learning with SoyBean Accuracy Results

Architectures	Optimizer	Dataset	Accuracy
MobileNet	Adam	SoyBean	83.49%
ResNet101	Adam	SoyBean	79.80%
ResNet50	Adam	SoyBean	79.77%
ResNet101	SGD	SoyBean	75.21%
ResNet18	Adam	SoyBean	72.06%
SqueezeNet	Adam	SoyBean	71.22%
ResNet50	SGD	SoyBean	57.83%
ResNet18	SGD	SoyBean	51.85%
MobileNet	SGD	SoyBean	51.44%
SqueezeNet	SGD	SoyBean	37.41%

Table 5. Transfer Learning with ASDID Accuracy Results

Architectures	Optimizer	Dataset	Accuracy
Squeezenet	Adam	ASDID	91.70%
ResNet101	Adam	ASDID	90.14%
MobileNet	Adam	ASDID	89.90%
ResNet50	Adam	ASDID	89.60%
ResNet18	Adam	ASDID	87.88%

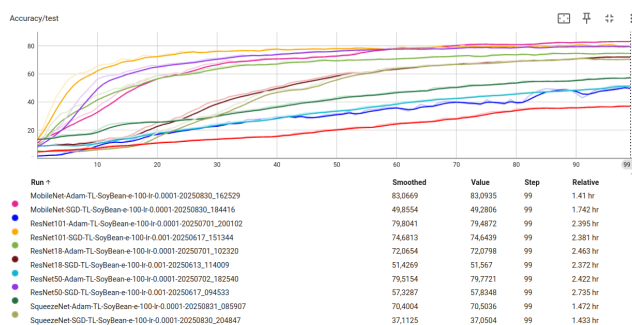


Figure 4. Accuracy curve on the test set in TensorBoard (Transfer Learning with SoyBean).

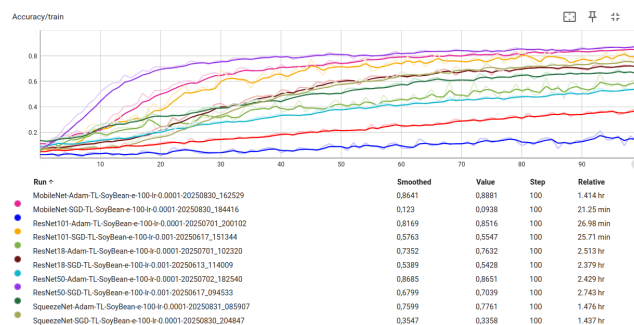


Figure 5. Accuracy curve on the train set in TensorBoard (Transfer Learning with SoyBean).

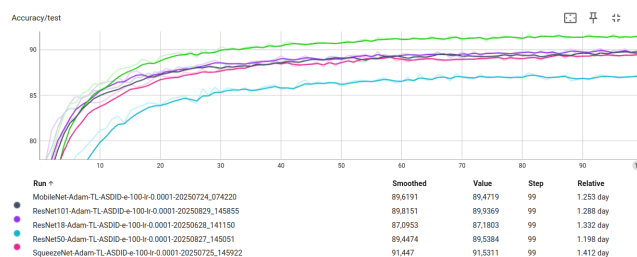


Figure 6. Accuracy curve on the test set in TensorBoard (Transfer Learning with ASDID).

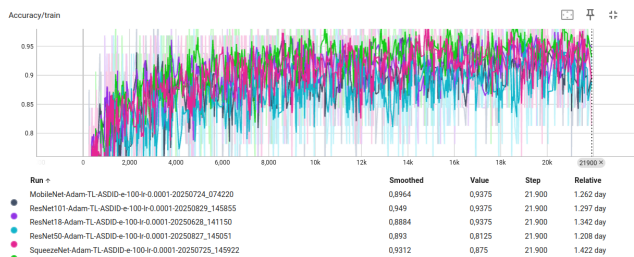


Figure 7. Accuracy curve on the train set in TensorBoard (Transfer Learning with ASDID).

5. Conclusion

Given the significant economic importance of soybean on the global scale and the need to improve productive quality, computational tools to perform an early diagnosis of leaf pathologies presents themselves as an ally. In this context, this work sought to analyze shallower and lighter neural networks than those commonly used larger CNN or Transformers

models, which would result in comparable quality in the end.

The experiments showed that, using fine-tuning and transfer learning techniques, the MobileNet and the SqueezeNet architectures, despite being more shallow, presented a performance comparable to that of deeper models. This finding supports the main proposition of this study: that efficient and accessible solutions for on-field disease diagnosis are possible and can also be highly effective. The presented results demonstrated that the accuracy achieved by lightweight models represents an excellent trade-off when compared to the high computational costs of heavier architectures.

Despite the promising results, our study has some limitations. The primary limitation is the use of images from a controlled environment for some diseases, which may not represent the great variability found in real-world conditions. Therefore, for future work it is necessary to train and validate these models on a larger and more diverse dataset of raw field images, including those with occlusion and even combined pathologies. Another interesting proposition for future development is the use of the diagnostic information provided by the application to track crop health generate statistical data. This data could support plant treatment and provide reports on the distribution and quantity of infected beings. Furthermore, exploring other lightweight architectures and implementing real-time segmentation methods could further refine the diagnostic process, moving beyond classification to precise disease location.

In conclusion, our work explored and validated the use of lightweight networks as an efficient and viable alternative for the automated diagnosis of soybean diseases. The reduction

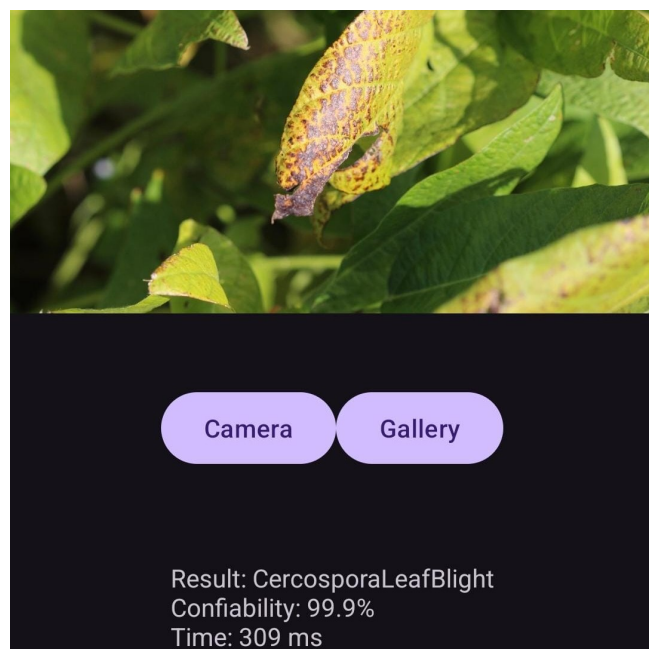


Figure 8. Android Application Layout

in computational costs, while maintaining a competitive level of accuracy, encourages the development of more accessible and scalable automated and technological tools to the field.

References

- [1] MOLINA, J. P. et al. Doenças da soja e perdas associadas no brasil e no mundo. In: *Embrapa Soja – Documentos 406*. Londrina, PR: Embrapa, 2019. p. 107–120. Disponível em: <https://www.alice.cnptia.embrapa.br/alice/bitstream/doc/11110682/1/aMolinap107.pdf>.
- [2] KP, A.; ANITHA, J. Plant disease classification using deep learning. In: *2021 3rd International Conference on Signal Processing and Communication (ICPSC)*. [S.l.: s.n.], 2021. p. 407–411.
- [3] BEVERS, N.; SIKORA, E. J.; HARDY, N. B. Soybean disease identification using original field images and transfer learning with convolutional neural networks. *Computers and Electronics in Agriculture*, v. 203, p. 107449, 2022. ISSN 0168-1699. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0168169922007578>.
- [4] PEYAL, H. I. et al. Plant disease classifier: Detection of dual-crop diseases using lightweight 2d cnn architecture. *IEEE Access*, v. 11, p. 110627–110643, 2023.
- [5] NG, H. F. et al. Plant disease detection mobile application development using deep learning. In: *2021 International Conference on Computer & Information Sciences (ICCOINS)*. [S.l.: s.n.], 2021. p. 34–38.
- [6] RAJPUT, A. S.; SHUKLA, S.; THAKUR, S. Soynet: A high-resolution indian soybean image dataset for leaf disease classification. *Data in Brief*, v. 49, p. 109447, 2023. ISSN 2352-3409. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2352340923005474>.
- [7] HE, K. et al. *Deep Residual Learning for Image Recognition*. 2015. Disponível em: <https://arxiv.org/abs/1512.03385>.
- [8] IANDOLA, F. N. et al. *SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and 0.5MB model size*. 2016. Disponível em: <https://arxiv.org/abs/1602.07360>.
- [9] HOWARD, A. G. et al. *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. 2017. Disponível em: <https://arxiv.org/abs/1704.04861>.
- [10] DENG, J. et al. Imagenet: A large-scale hierarchical image database. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, p. 248–255, 2009.